

- For every state, b is the branching factor if for all states, $|actions(state)| \leq b$
- Time complexity: number of nodes generated or expanded
- Space complexity: max number of nodes in memory

Breadth-First Search (BFS):

- Frontier: Queue
- Time complexity (# nodes generated): Exponential wrt depth of optimal solution
- Space complexity (size of frontier): Exponential wrt depth of optimal solution
- Complete: Yes, if b is finite
- Optimal: Yes, if step cost is same everywhere

Uniform-Cost Search (UCS):

- Frontier: Priority Queue (path cost)
- Time complexity: Exponential wrt "tier" of optimal solution
- Space complexity: Polynomial wrt max depth of search tree
- Complete: Yes, if positive step cost everywhere and finite total cost
- Optimal: Yes, if positive step cost everywhere

Depth-First Search (DFS):

- Frontier: Stack
- Time complexity: Exponential wrt max depth of search tree
- Space complexity: Polynomial wrt max depth of search tree
- Complete: No, when depth of search tree is infinite (action is reversible, may go back and forth)
- Optimal: No, optimal solution may be shallower depth

Search with visited memory:

- Applies to all, space complexity: $O(b^n)$
- Complete: Yes, if state space is finite
- Optimal: Not for DFS

A* Search:

- Frontier: Priority Queue $f(n)$
- Evaluation function $f(n) = g(n) + h(n)$, where $g(n) = \text{cost to reach } n$
- Time complexity: Exponential, good heuristic can improve
- Space complexity: Exponential
- Complete: Yes, if edge costs are positive and b is finite
- Optimal: Depends on heuristic

Depth-Limited Search (DLS):

- Limit search to depth l and backtrack when limit hit
- Time complexity: Exponential wrt depth limit
- Space complexity: Polynomial wrt depth limit if used with DFS
- Complete: No
- Optimal: No, if used with DFS

Iterative Deepening Search (IDS):

- Search with depth limit $0, 1, \dots$, return solution if found
- Time complexity: Exponential wrt depth of optimal solution (with overhead)
- Space complexity: Polynomial if used with DFS
- Complete: Yes
- Optimal: Yes, if step cost is the same everywhere

Heuristic:

- Estimate of optimal path cost from any state to goal state
- Admissible if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the optimal path cost to reach the goal state from n (never overestimate)
 - If $h(n)$ is admissible, A* search without visited memory is optimal
- Consistent if for every node n , every successor n' of n generated by any action a , $h(n) \leq c(n, a, n') + h(n')$, and $h(G) = 0$
 - Triangle inequality
 - If $h(n)$ is consistent, then $h(n)$ is admissible
 - If $h(n)$ is consistent, A* search with visited memory is optimal
 - If $h_1(n) \geq h_2(n)$ for all n , then h_1 dominates h_2
 - Dominant heuristic is better for search if admissible

Local Search (Hill Climbing Algorithm):

- Repeatedly move to the higher-valued successor until no better one exists
- **Adversarial Search (Minimax):**
 - Time complexity: Exponential wrt max depth $O(b^n)$, branch factor b , max depth m
 - Space complexity: Polynomial
 - Complete: Yes, if tree is finite
 - Optimal: Yes, against optimal opponent
 - Alpha-Beta pruning: Prune if $\alpha \geq \beta$
 - Time complexity: With perfect ordering, $O(b^{m/2})$
- Minimax with cutoff
 - Infeasible to evaluate entire tree
 - If terminal state, return utility(state), else heuristic for eval(state)
 - $h(\text{state})$ should fall between min and max utility
 - E.g. utility(loss) < $h(\text{state})$ < utility(win)
 - Admissibility and consistency do not apply
 - May be suboptimal wrt true minimax value

Machine Learning

- Classification tasks
 - Predict a discrete label/class/category from a set of predefined classes/categories based on input features
 - Output variable (target) is a categorical value
- Regression
 - Predict a continuous numerical value based on input features
 - Output variable (target) is a real number
- Performance measure for binary classification

$$\text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

If false **positives** are costly
e.g., spam prediction (missed emails)

Recall $R = \frac{TP}{TP+FN}$	If false negatives are costly e.g., cancer prediction (missed cancer)
F1 Score (harmonic mean between precision and recall) $F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}}$	

Balance precision & recall

- **Entropy** quantifies uncertainty or randomness of a random variable

Let Y be a random variable with outcomes values $y_1, \dots, y_k$ and probabilities $P(y_1), \dots, P(y_k)$. Then, the entropy is defined as:

$$H(Y) = H(P(y_1), \dots, P(y_k)) = - \sum_{i=1}^k P(y_i) \log_2 P(y_i)$$

- Binary outcomes with p positive and n negative outcomes:
 $P(\text{pos}) = p/(p+n)$, $P(\text{neg}) = n/(p+n)$
 $H(Y) = - \left(\frac{p}{p+n} \log_2 \frac{p}{p+n} \right) - \left(\frac{n}{p+n} \log_2 \frac{n}{p+n} \right)$

- **Specific conditional entropy** $H(Y|X=y)$ is the entropy of a random variable Y conditioned on the discrete random variable X taking a certain fixed value x

$$H(Y|X = x) = H(P(y_1|x), \dots, P(y_k|x))$$

- **Conditional entropy** $H(Y|X)$ quantifies the amount of uncertainty that remains about a random variable Y when another random variable X is known

$$H(Y|X) = \sum_{x \in X} P(x) H(Y|X = x)$$

- **Information gain** is a measure of reduction in uncertainty about a particular decision after knowing the value of a specific variable

$$IG(Y;X) = H(Y) - H(Y|X)$$

Uncertainty before Uncertainty after selecting a variable

- To build a decision tree, choose locally optimal attribute recursively to split at each step based on immediate information gain
- Pruning
 - Max depth: limit length of longest path from root node to leaf node
 - Min-sample leaves: set min threshold for number of samples required to be in a leaf node

Linear Regression

- Linear model is a function of the form:

$$h_w(x) = w_0x_0 + w_1x_1 + w_2x_2 + \dots + w_d x_d$$

- Minimises Mean Squared Error (MSE) loss:

$$J_{\text{MSE}}(w) = \frac{1}{2n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)})^2$$

- Normal equation:
 - Cost is d^n for inverting matrix
 - Will not work if $(X^T X)^{-1}$ is not invertible
 - Will not work for non-linear models

The best linear regression model is the one that has the **minimum loss**, which is when the **partial derivative**

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (x_j^{(i)} x^{(i)} - y^{(i)} x_j^{(i)}) = 0$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = 0 \quad \text{for all } w_j$$

Find the best w by solving a closed-form expression:

$$w = (X^T X)^{-1} X^T Y$$

- Gradient descent (differentiate wrt w)
 - For features of different scales:

$$w_j \leftarrow w_j - \gamma \frac{\partial J_{\text{MSE}}(w)}{\partial w_j}$$

Min-max scaling

$$x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$
 min(x_i) and max(x_i) denote the minimum and the maximum value of a feature x_i in dataset D .

Scales the features to be within [0,1].
 It is also common to scale the features to be within [-1,1]

Standardization

$$x'_i = \frac{x_i - \mu_i}{\sigma_i}$$
 μ_i and σ_i is the mean and the standard deviation of feature x_i in dataset D

This method transforms features to have a mean of 0 and a standard deviation of 1

Other solution: Different learning rate γ for each weight.

- Batch gradient descent: consider all training samples
- Mini-batch gradient descent: consider a subset of training examples at a time
 - Cheaper per iteration
 - Noisy gradient, may help escape local minima
- Stochastic gradient descent: select one random training example at a time
- Cheapest per iteration
- More noisy gradient, may help escape local minima/plateau

- Feature transformation
- **Polynomial features of degree k , $\phi_{pk}(x)$.**
 - Polynomial combinations of the features with degree $\leq k$.
 - When used with linear regression, this is called **polynomial regression**.
 - Example of 2-dimensional feature with degree-2 polynomial transformation:
 $\phi_{p2}([x_1 \ x_2]) \rightarrow [x_1 \ x_2 \ x_1 x_2 \ x_1^2 \ x_2^2]$

- **Log:** For each x_i , transform $x_i \rightarrow x_i$ and $\log(x_i)$.
 - Example: $\phi_{\log}([x_1 \ x_2]) \rightarrow [x_1 \ x_2 \ \log(x_1) \ \log(x_2)]$
- **Exponential:** For each x_i , transform $x_i \rightarrow x_i$ and $\exp(x_i)$.
 - Example: $\phi_{\exp}([x_1 \ x_2]) \rightarrow [x_1 \ x_2 \ \exp(x_1) \ \exp(x_2)]$

Logistic Regression

Logistic Function:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

$$h_w(x) = \sigma(w_0x_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d)$$

Definitions:

- **Cross entropy** measures the difference between two probability distributions over the same set of outcomes
 - P represents the true probability distribution
 - Q represents a prediction of the probability of the outcomes

$$H(P, Q) = - \sum_x P(x) \log(Q(x))$$

- **Binary Cross Entropy (BCE)** is a special case of cross entropy with only two outcomes

$$-P(1) \log(Q(1)) - P(0) \log(Q(0))$$

The "true" distribution P can be described by a single value y , the true label (1 or 0). The predicted distribution Q can be described by a single value $\hat{y}$, which is the model's predicted probability that the outcome is class 1. The probability for class 0 is $1 - \hat{y}$.

$$BCE(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

Loss Function:

$$J_{\text{BCE}}(w) = \frac{1}{n} \sum_{i=1}^n BCE(y^{(i)}, h_w(x^{(i)}))$$

- **Partial Derivative:**

$$\frac{\partial J_{\text{BCE}}(w)}{\partial w_j} = \frac{\partial}{\partial w_j} \frac{1}{n} \sum_{i=1}^n BCE(y^{(i)}, h_w(x^{(i)}))$$

$$= \frac{1}{n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- Linearly separable data
 - When a single linear decision boundary separates the different classes
 - Requires non-linear feature transformations if not (poly, log, exp)

- Binary to Multi-class classification
 - One vs One: one classifier for every pair
 - $K(K-1)/2$ binary classifiers
 - Each classifier votes for a class, and the class with most votes is selected
 - One vs Rest: for each class, treat all other classes as a single combined class
 - K classifiers
 - Classifier with the highest probability output determines class

- Multi-label classification
 - Transformation to binary classification problem
 - **Binary Relevance:** Train one independent binary classifier per label (One-vs-Rest). Ignores label correlations and constraints (e.g., co-occurrence, mutual exclusivity).
Example: scene classifier $\rightarrow$ mountain classifier, field classifier
 - **Classifier Chain:** Like Binary Relevance but feed predictions of previous labels as features for subsequent labels. Captures label dependencies, but sensitive to chain order.

The idea of relevance can be extended to non-binary classifiers.
Example: medical diagnosis $\rightarrow$ diabetes level [1,2,3,4], blood pressure [low, normal, high].

- Transformation into multi-class classification problem
 - **Label Posenet:** Treat each unique label-set as a single multiclass label. Preserves correlations but can explode in classes and be data sparse.
Example: scene classifier predicts "mountain", "field", "mountain and field", ...

- Hyperparameters are settings that control the behaviour of the training algorithm and model but not learned from data
 - Set before training process begins (predefined and adjusted manually while parameters are learned during training)
 - Hyperparameter tuning techniques: grid search (exhaustive search), random search, local search, successive halving, Bayesian optimization, ...

- **Regularization, Kernels and Support Vector Machines (SVM)**

- Overfitting often associated with large-magnitude weights
- Augment the loss function $J(w)$ with a penalty function or regularizer $R(w)$, which is often (not always) defined using an L_p -norm

For $p \geq 1$ and a vector w of dimension $d+1$, $w = [w_0, w_1, \dots, w_d]^T$, the L_p norm is defined as:

$$\|w\|_p = \left(\sum_{i=1}^d |w_i|^p \right)^{\frac{1}{p}}$$

- $\lambda \geq 0$ is the regularization parameter which controls the strength of the penalty. The regularized loss function is as follows:

$$J_{\text{reg}}(w) = J(w) + \lambda R(w)$$

- **Derivative:**

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \frac{\partial w_j}{\partial w_j} \lambda$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

$$\frac{\partial J_{\text{MSE}}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^{(i)} x^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

- $X^T X + \lambda I$ is invertible for any real number $\lambda > 0$
- For any $\lambda > 0$, the ridge-regression has a unique closed form solution

- For any $\lambda > 0$, the L_2 norm of the L_2 regularized weight vector is strictly smaller than the non-regularized weight vector, assuming the solution is non-zero

- L_1 regularizer will penalize the features with a constant penalty of λ (in terms of gradient) if a feature has non-zero weight

For a feature to be included in the model, its contribution to reducing the MSE loss must be **exactly equal** to the regularization penalty λ (in terms of gradient).

If a feature's weight is **less than** the penalty λ , the optimization will prefer to set its weight to **exactly zero** rather than pay the L_1 penalty, inducing sparsity.

Kernels:

- Feature transformation can lead to large number of weights required

$$\text{In general, for } d \text{ features and degree } k \text{ transformation: } \binom{d+k}{k}$$

- For a linear model parameterized with a weight vector w whose prediction is of the form $w^T x$, where f is a function, the best weight vector w that minimizes certain regularized loss $J_{\text{reg}}(w)$ can be expressed as:

$$w = \sum_{i=1}^n a_i x^{(i)}$$

where α is a vector of real numbers $\alpha_1, \alpha_2, \dots, \alpha_n$ of dimension n

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ 1 & x_1^{(n)} & x_2^{(n)} \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(n)} \end{bmatrix}$$

Using the push-through matrix identity on the expression given by the normal equation:
 $(X^T X + \lambda I)^{-1} X^T = X^T (X X^T + \lambda I)^{-1}$
 where X is a matrix, I identity matrix, and λ is a non-zero scalar.

Here, $(X X^T + \lambda I)^{-1} Y$ is an n -dimensional vector. Let's call this vector α .

This is a special form/application of the **Woodbury matrix identity**.

- As a consequence, linear model can be expressed using the α vector as follows:

$$h_w(x) = w^T x = \sum_{i=1}^n \alpha_i x^{(i)T} x = h_\alpha(x)$$

The model:
 $h_w(x)$ is called the **primal formulation of the model**.
 $h_\alpha(x)$ is called the **dual formulation of the model**.
 • Optimal model: primal = dual.

The same applies to logistic model (due to the representer theorem).

For ridge regression, we had the following objective optimizing for w ; this is called the **primal objective**.

$$J_{\text{reg}}^{\text{MSE}}(w) = \frac{1}{2n} \sum_{i=1}^n (w^{(i)T} x^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^d w_j^2$$

For the dual formulation, we optimize the **dual objective** instead, i.e., minimize the loss over α :

$$J_{\text{reg}}^{\text{MSE}}(\alpha) = \frac{1}{2n} \sum_{i=1}^n \left(\sum_{j=1}^n \alpha_j x_j^{(i)T} x^{(i)} - y^{(i)} \right)^2 + \lambda \sum_{j=1}^n \alpha_j^2$$

Theorem: Minimizing $J_{\text{reg}}^{\text{MSE}}(\alpha)$ is equivalent to minimizing $J_{\text{reg}}^{\text{MSE}}(w)$.

- $h_\alpha(x)$ has n weights and is dependent on number of training samples
- The dual formulation can be written more generally using inner product as follows:

$$h_w(x) = \sum_{i=1}^n \alpha_i x^{(i)T} x = \sum_{i=1}^n \alpha_i \langle x^{(i)}, x \rangle$$

Theorem (Informal): For any feature mapping function ϕ , there must exist a function $h_\alpha(u, v)$ that computes $\langle \phi(u), \phi(v) \rangle$, for any vectors u and v . We call this function **kernel**.

Suppose that you use the transformed feature $\phi(x)$ as input, your model becomes

$$h_\alpha(\phi(x)) = \sum_{i=1}^n \alpha_i \langle \phi(x^{(i)}), \phi(x) \rangle = \sum_{i=1}^n \alpha_i k(x^{(i)}, x) = h_\alpha^k(x)$$

- Do not have to expand $\phi(x)$ explicitly; simply compute kernel between $x^{(i)}$ and x . This is called kernel trick.
- **Polynomial degree k kernel**
 - **Feature map:** Polynomial combinations of the features with degree $\leq k$.
Example (

